上位机监控接口JSON定义V1.2

一、主Json结构（主从模式通用）

主Json

```
{
  "command_id": 1,
  "data": {}
}
```

字段	类型	必选	说明
command_id	int	是	1=get_status, 2=manu, 3=send_msg, (4=set_parameter 感觉远程修改参数有风险，建议暂不实现)
data	object	是	具体命令对应的数据

二、具体命令对应的子Json设计

0.对于不符合格式的请求返回json

代码块

```
{
  "command_id": 1,
  "code": 1,
  "message": ""     //表示错误原因
}
```

1.get_status - 获取设备状态（对应需求1,2,3,6,9）

服务器请求：

代码块

```
{
```

```
  "command_id": 1,
  "data": null
}
```

上位机响应：

代码块

```
{
  "command_id": 1,
  "code": 0,
  "data": {
    "device_id": "SLM_001",
    "priting_status": "printing",
    "time": {
      "start_time": "11:12:10",
      "elapsed_seconds": "00:00:10",
      "remaining_seconds": "10:12:00"
    },
    "part_info": {
      "jobs": [
        {
          "name": "零件1.slc",
          "is_canceled": false,
          "layer_count": 500,
          "current_layer": 150
        },
        {
          "name": "零件2.slc",
          "is_canceled": false,
          "layer_count": 300,
          "current_layer": 0
        }
      ]
    },
    "powder": {
      "remaining_powder": 15.5,
      "status": "sufficient",  // "sufficient", "warning", "insufficient"
      "powder_supply_amount": 0.8
    },
    "machine_status": {
      "wind_speed": 1.2,
      "temperature": 25.6,
      "oxygen_content": 0.01,
      "pressure": 10.0,
      "unit": {
```

```
          "wind_speed": "m/s",
          "temperature": "°C",
          "oxygen_content": "%",
          "pressure": "kPa"
        }
      },
      "alarms": [
        {
          "level": "warning",
          "message": "温度接近上限",
        }
      ]
    }
  }
```

1. 根对象

字段	类型	必选	说明
command_id	int	是	指令ID，1表示获取设备状态
code	int	是	响应码，0表示成功，1表示拒绝执行
data	object	是	设备状态数据容器

2. data对象

字段	类型	必选	说明
device_id	string	是	设备名称，标识符

priting_status	string	是	打印状态
			Printing = 0, // 打印中
			Paused = 1, // 暂停
			Stop = 2, // 停止

time	object	是	时间相关信息
partNums	string	是	零件个数
part_info	object	是	零件信息
powder	object	是	粉量状态
machine_status	object	是	机台状态数据 下面第6点有解释说明
alarms	array	是	报警信息列表

3. time对象

字段	类型	必选	说明
start_time	string	是	开始打印时间
elapsed_seconds	string	是	已打印时长
remaining_seconds	string	是	剩余打印时间

4. part_info对象

字段	类型	必选	说明
jobs	array	是	打印任务列表

jobs数组元素

字段	类型	必选	说明
name	string	是	工件文件名
part_id	int	是	工件名唯一id
is_canceled	bool	是	是否已取消
layer_count	int	是	总层数
current_layer	int	是	当前层数

5. powder对象

字段	类型	必选	说明
remaining_powder	float	是	剩余粉量
status	string	是	粉量状态0, // 充足 Sufficient 1, // 警告 Warning 2 // 不足 Insufficient
powder_supply_amount	float	是	供粉量

6. machine_status对象

字段	类型	必选	说明
wind_speed	float	是	风速(m/s)
temperature	float	是	温度(°C)
oxygen_content	float	是	氧含量(%)
pressure	float	是	压力(kPa)
supMachineTemperature	float	是	辅机温度(°C)
supMachinePressure	float	是	辅机压力压力(kPa)
windMachineFrequence	float	是	风机频率(Hz)
supMachineOxygenContent	float	是	辅机氧含量(%)
unit	object	是	单位定义

unit对象

字段	类型	必选	说明
wind_speed	string	是	风速单位
temperature	string	是	温度单位
oxygen_content	string	是	氧含量单位
pressure	string	是	压力单位
supMachineTemperature	string	是	辅机温度单位
supMachinePressure	string	是	辅机压力压力单位
windMachineFrequence	string	是	风机频率单位
supMachineOxygenContent	string	是	辅机氧含量单位

7. alarms数组元素

字段	类型	必选	说明

level	string	是	报警级别，
			Normal = 0, // 普通消息
			Important = 1, // 重要消息 如排班计划
			Warning = 2, // 警告，需要操作员注意
			Error = 3, // 错误,需要暂停
			Fault = 4 // 故障,需要急停

message	string	是	报警信息

2：manu-制造控制命令 (对应需求5)

服务器请求：

代码块

```
{
  "command_id": 2,
  "data": {
    "action": 1,
    "device_id": "SLM_001",
    "job_name": "//print.fd"
  }
}
```

字段	类型	必选	说明
command_id	int	是	指令id,2表示制造控制指令

action	int	是	操作码 enum class ManuAction {
			Prepare = 1, // 准备制造
			Pause = 2, // 暂停
			Stop = 3 // 终止
			};

device_id	string	是	设备名称，标识符每台机器不一样
job_name	string	是	共享文件夹中的工程文件路径名

上位机响应：

代码块

```
{
  "command_id": 2,
  "data": {
    "code": 1,
    "device_id": "SLM_001"
  }
}
```

字段	类型	必选	说明
command_id	int	是	指令id,2表示制造控制指令
code	int	是	响应码，0表示成功，1表示拒绝执行
device_id	string	是	设备名称，标识符

3：send_msg-发送消息(对应需求8)

服务器请求：

代码块

```
{
  "command_id": 3,
  "data": {
    "id": 1,
    "device_id": "SLM_001",
    "type": "Important",
    "content": "2023年10月生产排班",
    "job_name": "C//print.fd"

  }
}

```

字段	类型	必选	说明
command_id	int	是	指令id,3表示发送消息指令
id	int	是	操作码，1代表工作安排
device_id	string	是	设备名称，标识符

type	string	是	消息类型，重要、警告
			Normal = 0, // 普通消息
			Important = 1, // 重要消息 如排班计划
			Warning = 2, // 警告，需要操作员注意
			Error = 3, // 错误,需要暂停
			Fault = 4 // 故障,需要急停

content	string	是	消息内容，如后续排班信息；
job_name	string	是	fd文件路径

上位机响应：

代码块

```
{
  "command_id": 3,
  "code": 0,
  "data": null
}
```

字段	类型	必选	说明
command_id	int	是	指令ID，3表示发送消息指令
code	int	是	响应码，0表示成功，1表示拒绝执行

4：set_parameter-修改参数

服务器请求：

1.修改供粉量

代码块

```
{
  "command_id": 4,
  "data": {
    "action": 1,
    "device_id": "SLM_001",
    "powder": 1.25
  }
}
```

2.取消或者加入打印零件

代码块

```
{
  "command_id": 4,
  "data": {
    "action": 2,
    "device_id": "SLM_001",
    "cancel_parts_id": 1
  }
}
```

2.取消或者加入打印零件

字段	类型	必选	说明
command_id	int	是	指令id,4表示修改参数指令
action	int	是	1:修改全局参数 2：修改零件参数(预留，未实现)
device_id	string	是	设备名称，标识符
powder	float	否	供粉量
cancel_parts_id	int	否	取消打印零件id,对应action=2(若原零件处于打印状态，该零件取消打印，反之则加入打印)

上位机响应：

代码块

```
{
  "command_id": 4,
  "data": {
    "code": 1,
    "device_id": "SLM_001"
  }
}
```

字段	类型	必选	说明
command_id	int	是	指令id,4表示修改参数指令
code	int	是	响应码，0表示成功，1表示拒绝执行
device_id	string	是	设备名称，标识符

三、C++代码段示意

1.枚举类

代码块

```
/**
 * @brief 打印状态枚举
 */
enum class PrintStatus {
```

```
    Printing = 0,     // 打印中
    Paused = 1,       // 暂停
    Stop = 2,      // 停止
};

/**
 * @brief 粉量状态枚举
 */
enum class PowderStatus {
    Sufficient = 0,  // 充足
    Warning = 1,      // 警告
    Insufficient = 2 // 不足
};

/**
 * @brief 消息级别枚举  供报警和发送消息使用
 */
enum class MsgLevel {
    Normal = 0,       // 普通消息
    Important = 1,            // 重要消息  如排班计划
    Warning = 2,      // 警告，需要操作员注意
    Error = 3,      // 错误,需要暂停
    Fault = 4      // 故障,需要急停
};

/**
 * @brief 制造控制动作枚举
 */
enum class ManuAction {
    Prepare = 1, // 准备制造
    Pause = 2,         // 暂停
    Stop = 3          // 终止
};

```

2.使用示例

代码块

```
// 使用nlohmann/json库的示例
#include <nlohmann/json.hpp>
using json = nlohmann::json;

// 解析主命令
json received_command = json::parse(received_data);
int command_id = received_command.at("command_id").get<int>();
```

```
8
9   switch (command_id) {
10      case 1: { // get_status
11          json response;
12          response["command_id"] = 1;
13          response["code"] = 0;
14          response["message"] = "success";
15
16          json time_data;
17          time_data["start_time"] = get_start_time();
18          time_data["elapsed_seconds"] = get_elapsed_time();
19          time_data["remaining_seconds"] = get_remaining_time();
20
21          json data;
22          data["time"] = time_data;
23          // ... 填充其他数据
24
25          response["data"] = data;
26          return response.dump();
27      }
28      // ... 其他命令处理
29  }
```